Development and Validation of a Digital Twin for a Commercial Mobile Robot

1th Wilson Eduardo Barioni
RCPGEI - UTFPR
Curitiba, Parana, Brazil
eduardo.barioni@agvs.com.br

2th Igor Pardal Latini
CPGEI - UTFPR
Curitiba, Parana, Brazil
igorlatini@alunos.utfpr.edu.br

3th Carina Toledo Scoparo Barioni
Universidade Positivo
Paraná, Brasil
scoparo@gmail.com

4th Flavio Neves-Jr
CPGEI - UTFPR
Curitiba, Parana, Brazil
neves@utfpr.edu.br

5th L.V.R. Arruda (IEEE member)
CPGEI - UTFPR
Curitiba, Parana, Brazil
lvrarruda@utfpr.edu.br

6th Marco Teixeira
PPGIa - PUCPR
Curitiba, Parana, Brazil
simoes.marco@ppgia.pucpr.br

2024 Eighth IEEE International Conference on Robotic Computing (IRC) | 979-8-3315-2155-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/IRC63610.2024.00042

***Abstract*—In recent decades, there has been a significant increase in the presence of robots in the daily operations of industries. Automated Guided Vehicles (AGVs) used in industrial environments are gaining prominence due to their numerous applications, ranging from logistic tasks such as material transport to more complex process tasks like replacing conveyor belts in production lines. For a mobile robot to operate in these environments, it must possess skills to perform its functions and ensure safety. In this regard, simulating a computational model of a real autonomous system can contribute to developing new operational strategies. The objective of this work is to explore the use of a Digital Twin for an Automated Guided Vehicle (AGV), aiming to design and validate navigation and obstacle avoidance algorithms, with the intent to optimize the operation of these devices in industrial environments. In the current study, the AGV G3R1000 was used, which was modeled in detail in FUSION 360 software, along with a production line from the AGVS company in Brazil. As validation, the digital robot was used to design the navigation software using the PID control system and obstacle avoidance using Fuzzy control. The algorithm developed in the digital robot was applied to the real robot, and a performance comparison was made. In conclusion, it is possible to use the simulated robot to design algorithms for the real robot, saving resources such as time and equipment.**

AGV, PID, Fuzzy, CoppeliaSim, Simulation

I. Introduction

The Fourth Industrial Revolution, commonly referred to as Industry 4.0, heralds a period of significant advancements in how production systems incorporate cutting-edge digital technologies, leading to an unprecedented transformation in manufacturing processes. Among these innovations, Automated Guided Vehicles (AGVs) and Autonomous Mobile Robots (AMRs) stand out for their ability to enhance efficiency and logistical adaptability within industrial settings [1]; [2]. Nevertheless, the uptake of these technologies faces notable hurdles, such as high initial expenses and intricate integration challenges, which may impede their adoption across companies of varying scales. In light of these obstacles, Digital Twins emerge as an innovative solution, providing precise virtual models that replicate physical systems, thus facilitating simulations, testing, and optimization both prior to and during AGV implementation [3]. This approach holds the potential to significantly reduce costs and implementation time while simultaneously bolstering the safety and efficiency of automated transportation systems. Existing literature already showcases the practicality and advantages of Digital Twins across diverse industrial contexts, presenting ample opportunities for exploration in optimizing both AGVs and AMRs [4]; [5]. Therefore, te intricacies of modern manufacturing environments and the imperative to integrate with legacy systems pose notable challenges. Effectively implementing Digital Twins offers a feasible solution, enabling simulation and optimization of AGV systems in virtual environments prior to physical execution, thereby mitigating potentially costly errors and delays in the production schedule. Thus, successful deployment of Digital Twins for AGVs necessitates a comprehensive understanding of the involved technologies, the industrial processes slated for optimization, and the methodologies governing the development and deployment of these digital systems. Essentially, this entails real-time data integration, process modeling, and the simulation of operations and failure scenarios—critical components for preempting challenges and fine-tuning AGV performance in complex industrial environments [6]. Therefore, the aim of this article is to explore the intersection between Digital Twins and AGVs in Industry 4.0, proposing a methodology to minimize time and implementation costs of these systems. Through a multidisciplinary approach encompassing theories of information systems, software engineering, and operations management, a computational model of a real autonomous system was simulated using control software, aiming to design algorithms for the real robot. With this, we aim to make a significant contribution by providing practical insights to the

Graduate Program in Electrical and Computer Engineering (CPGEI), Universidade Tecnologica Federal do Parana (UTFPR), Av. Sete de Setembro, 3165, Curitiba, Parana, Brazil

Graduate Program in Informatics (PPGIa), Pontifícia Universidade Católica do Paraná (PUCPR), Rua Imaculada Conceição, 1155, 80215-901 Curitiba, Brazil

979-8-3315-2155-4/24/$31.00 ©2024 IEEE
DOI 10.1109/IRC63610.2024.00042

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.

industry on how to maximize the potential of Digital Twins in optimizing AGV systems.

II. Methodology

A. Computational Model Structure

The development of the work was carried out using the Windows operating system, where the Fusion 360 software is available for creating three-dimensional models of the production line and the AGV. The Linux operating system was used for creating the simulation scenario, which was modeled using CoppeliaSim software programmed in LUA and the ROS package to control the AGV programmed in C++.

B. Fusion 360

The three-dimensional modeling of the production line and the AGV was developed using Autodesk's Fusion 360 software. The choice of software was due to the availability of a free version for students. After the modeling was completed, the files were exported in STL format, a format accepted by the CoppeliaSim simulation software. Figure 1 shows how the files are exported in STL (Binary) format in Fusion 360.

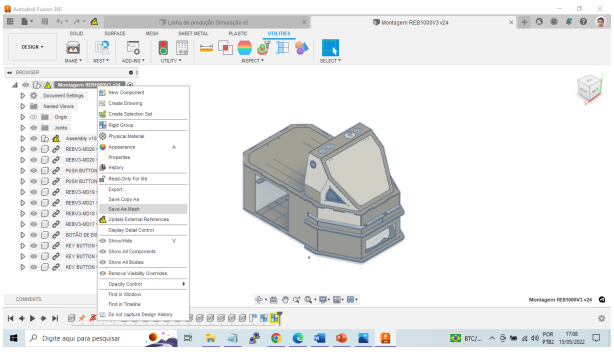


Fig. 1. Export in STL format

To ensure greater realism in the simulation, the AGV was exported in two separate parts: the fairing and the drive for locomotion and steering. The two rear wheels and two front wheels were removed before exporting, and later created within CoppeliaSim. To facilitate assembly in the simulated environment within CoppeliaSim, the components of the production line were combined, thus becoming a single component and subsequently exported in STL (Binary) format.

C. CoppeliaSim

CoppeliaSim is a robotics simulation software created by Coppelia Robotics, where it is possible to computationally simulate, through kinematic and dynamic models, the behavior of a robot in a virtual environment, without the need to actually build the robot [7].

To create the scene within the simulation environment, the first step was to import the production line, fairing, and drive using the "mesh" import tool, as illustrated in Figure 2. Figure 3 shows the production line imported into the virtual environment. The components in STL format are composed of several triangular geometric shapes, a format that requires CoppeliaSim to use a lot of processing power. Therefore, aiming for better performance, the editing tool, which can be observed in Figure 4, was used to join the triangles and form a new structure of the production line, fairing, and drive within CoppeliaSim, ensuring the original dimensions created in Fusion 360.

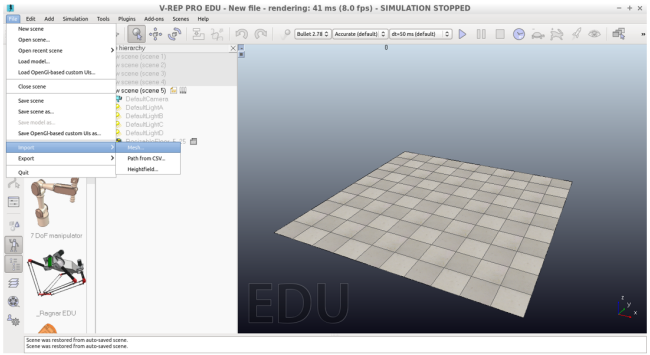


Fig. 2. Importing model into the simulation environment

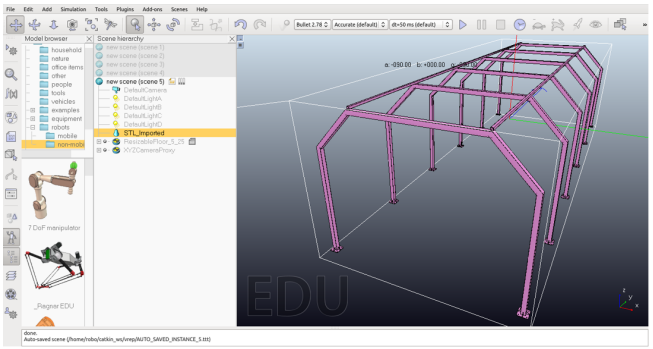


Fig. 3. Completion of the production line import

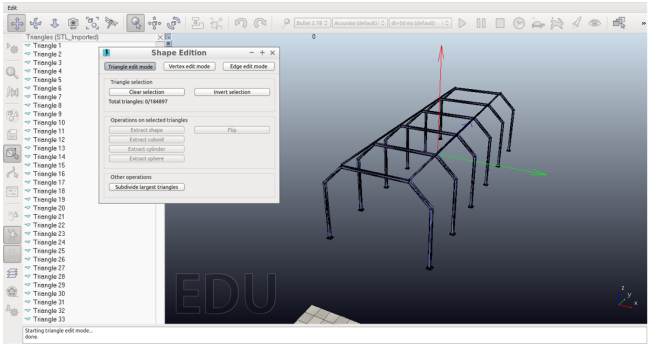


Fig. 4. Editing the production line

To complete the necessary components for the AGV, the Hokuyo laser scanner was imported as shown in Figure 5.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.

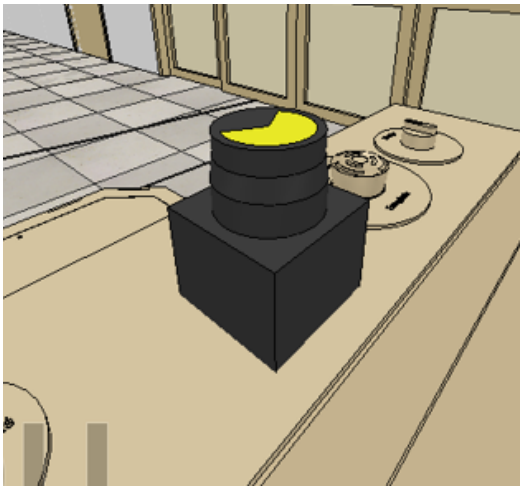


Fig. 5. Hokuyo Laser Scanner

With all components imported into the simulation environment, it was possible to develop the second stage, the assembly of the AGV. To begin assembling the robot, the drive was positioned inside the fairing, with the creation of missing components, such as the rear and front wheels, the support for the laser scanner, and the navigation antenna, which were positioned in their respective places.

The third stage of assembly was marked by the creation of joints, as can be observed in Figure 6. In the case of the AGV, seven revolute joints were needed, with two configured as motors in the two front wheels, three free joints, being in the two rear wheels and drive for locomotion, one for fixing the antenna, and one for fixing the scanner support. The joints can be observed in Figure 7.

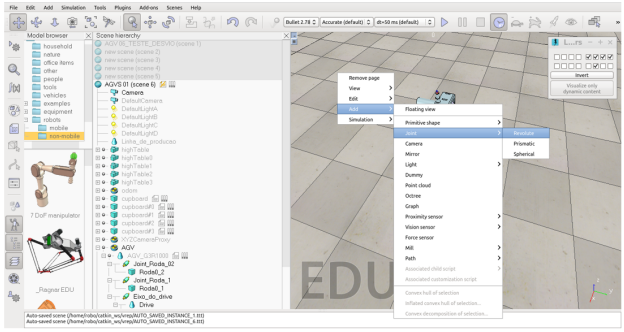


Fig. 6. Creation of Joints

To ensure a realistic scenario, data were collected from the floor plan of the production line area, thus making it possible to add walls, doors, windows, cabinets, and tables to the scene, as can be observed in Figure 8.

D. Robotic Operating System (ROS)

The Robot Operating System (ROS) is a set of software libraries and tools that help in building robot applications [8].

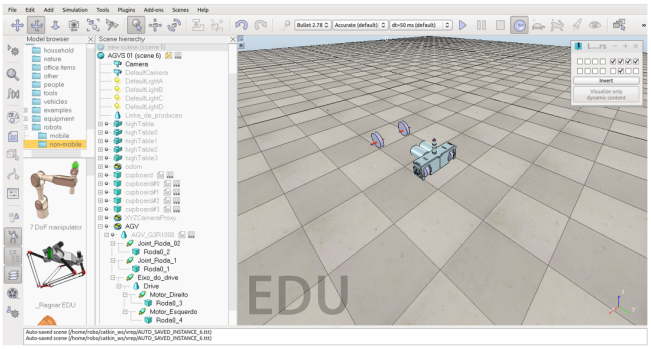


Fig. 7. View of the AGV without the fairing, highlighting the joints

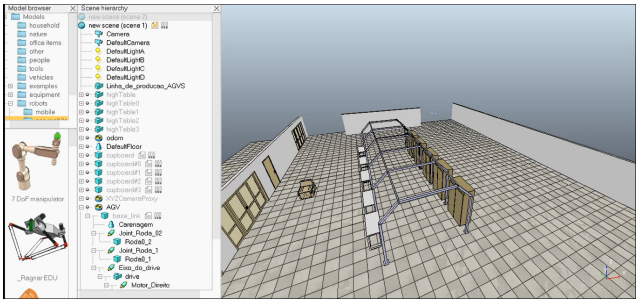


Fig. 8. Dimensions of the production line

The CoppeliaSim scene communicates using the concept of nodes and topics present in ROS. For this work, a node in C++ was created, which is responsible for all the motion control of the AGV, and a script in LUA in CoppeliaSim, which has the function of publishing the odometry of the antenna and scanner, as well as reading the topic of speed control of the AGV's motors. Figure 9 illustrates the architecture of the system, showing the interaction between the systems, as well as the structure of topics used."

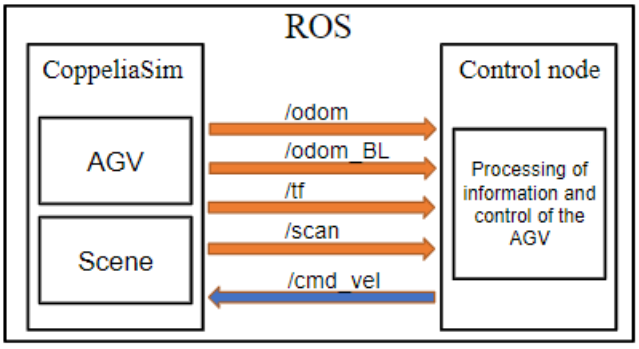


Fig. 9. System Architecture

The types of messages used in ROS were:

```
std_msgs - String
geometry_msgs - Twist
sensor_msgs - LaserScan
```


Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.

`nav_msgs - Odometry`

E. AGV Path

The path of the AGV was defined based on the point-following technique, where the AGV follows the next point each time it reaches one. The steering angle is controlled by a PID controller, which calculates the error through the difference between the angle formed by the vector between the current position of the vehicle and the next planned point on the path. For the course, 8 points were selected being: P1(x=4.5, y=0); P2(x=6.5, y=-4); P3(x=6.5, y=-8); P4(x=4.5, y=-9); P5(x=-6.5, y=-9); P6(x=-8.5, y=-8); P7(x=-8.5, y=-2); P8(x=-6.5, y=0), as can be seen in Figure 10.

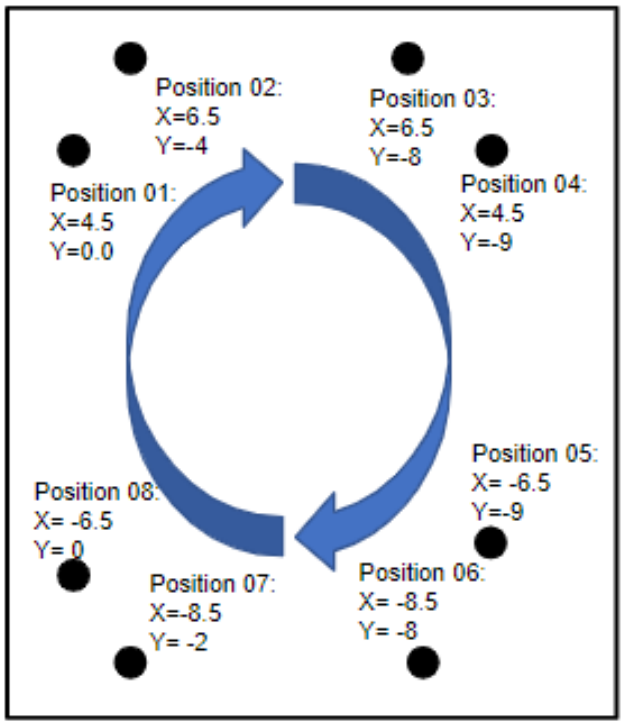


Fig. 10. Route

The PID controller (Proportional Integral Derivative) is a control technique that combines proportional, integral, and derivative actions, with the purpose of minimizing the error signal.

Equation 1 represents the PID controller equation as a function of time.

$$u(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de(t)}{dt} \qquad (1)$$

F. Obstacle Avoidance

To perform obstacle avoidance, a fuzzy controller was used. For the implementation of the fuzzy controller in the main ROS node, the FuzzyLite library from ROS was utilized. The inputs for the fuzzy controller are the distance data and the angle of the drive in relation to the fairing of the AGV, and the outputs are the speed control and the steering angle of the movement, as can be observed in Figure 11.

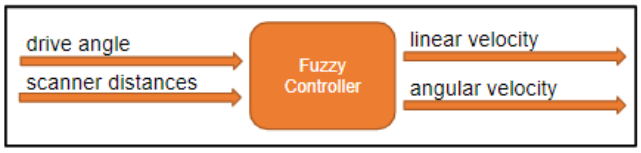


Fig. 11. Fuzzy Controller

The Hokuyo laser scanner provides 512 distance measurements over 180 degrees. As input for the fuzzy controller, it was decided to divide the scanner into 6 equal parts, each part corresponding to 30 degrees and behaving as a single sensor, using only the measurement of the shortest distance. The sensors were named 'Far Right,' 'Right,' 'Center Right,' 'Center Left,' 'Left,' and 'Far Left,' as represented in Figure 12. This way, the fuzzy controller can determine whether the obstacle is to its right or left.

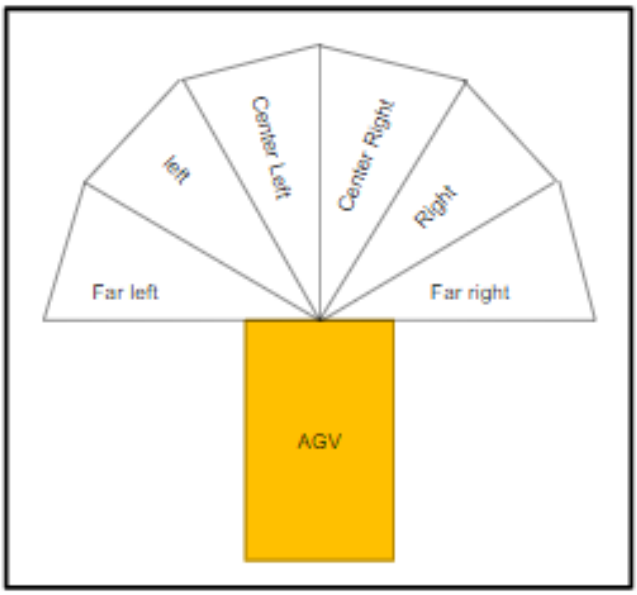


Fig. 12. Fuzzy Controller

The distances from each of the 6 sensors are also considered by the fuzzy controller, in this way, when an obstacle is detected, it is possible to know if it is "Near", "Close", or "Far". If the obstacle is detected at less than 1 meter, it is considered near, less than 3 meters is considered close, and above 3 meters is considered far. The membership functions associated with these predicates are given in Figure 13.

Based on the distance values of the six sensors, the fuzzy controller calculates the linear velocity and the angular velocity to correctly avoid the obstacle. The rules for controlling the linear velocity represent the following analysis: the closer the vehicle is to the obstacle, the lower the linear velocity

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.

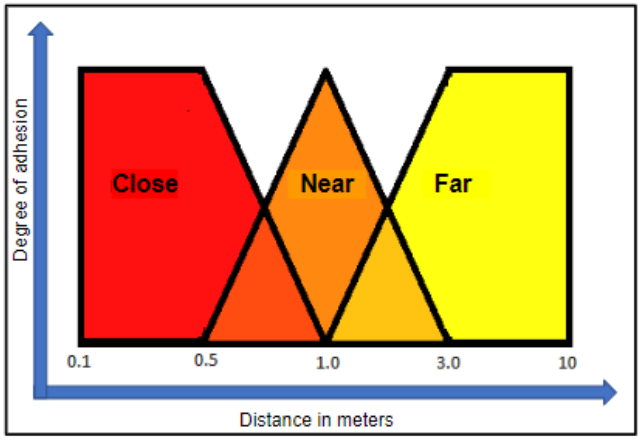


Fig. 13. Input Membership Function

sent to the AGV. If it gets too close to an obstacle at a distance where it would be impossible to avoid, the AGV stops completely. The linear velocity is controlled at three levels: 'Slow', 'Normal', 'Fast', as per the membership functions given in Figure 14.

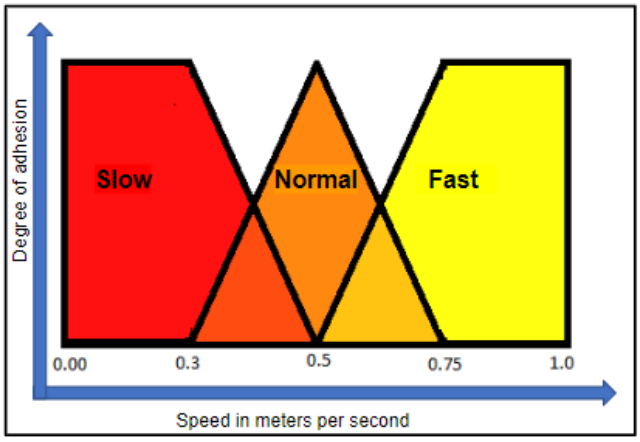


Fig. 14. Output Membership Functions: Linear Velocity

The rules for linear velocity control are intuitive; some examples include: if it's far away, do not reduce speed; if it's close, reduce to normal; and if it's very close, reduce to the maximum. The rules for linear velocity control are represented in Figure 15.

The operation of the angular velocity control is similar to that of linear velocity control, with the sensor indicating the angle and distance of the obstacle. The predicates used in steering are classified as 'Far Right', 'Right', 'Center Right', 'Center Left', 'Left', and 'Far Left', as illustrated in Figure 16.

The AGV model used in the study has a differential drive locomotion system, represented in Figure 17, and the sensors should be related to the angle of the locomotion drive. There-

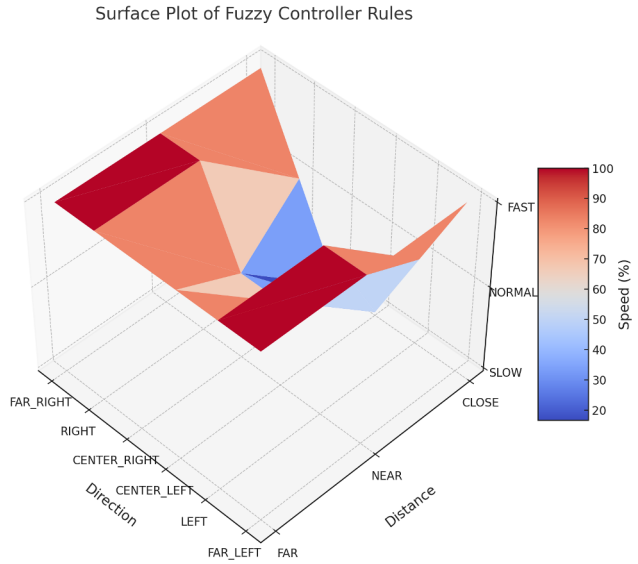


Fig. 15. Output Membership Functions: Linear Velocity

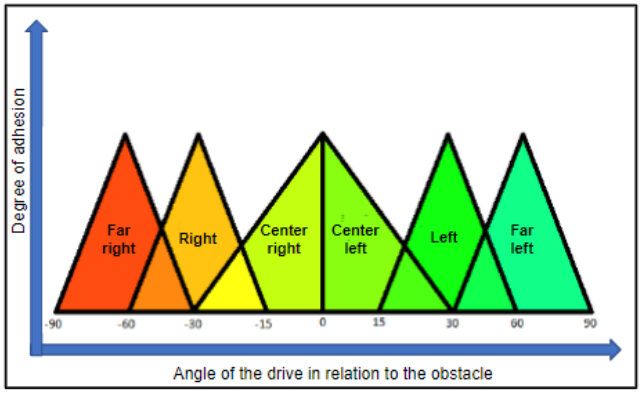


Fig. 16. Output Membership Functions: Linear Velocity

fore, it was necessary to calculate the transformation of the angle of the laser scanner with the angle of the locomotion drive. This transformation is detailed in Figure 18.

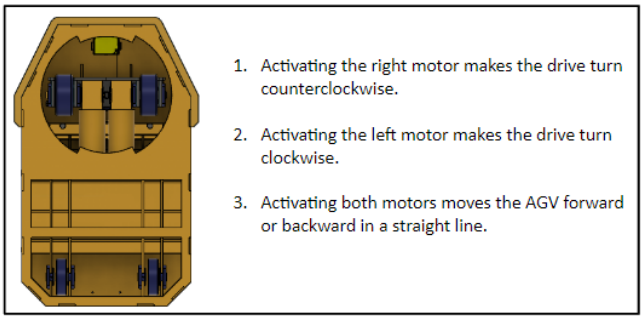


Fig. 17. Differential Type Locomotion

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.

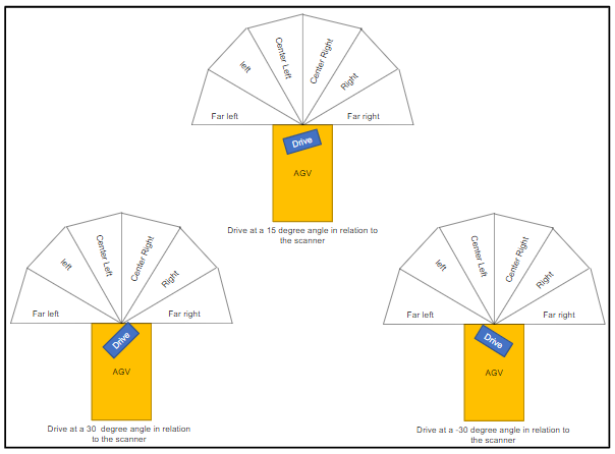


Fig. 18. Transformation of the Scanner Relative to the Locomotion Drive

To perform the calculation of angular velocity control, the fuzzy controller uses the sensor that shows the shortest distance to the obstacle. If the sensor with the shortest distance is the right sensor, the fuzzy controller corrects the angular velocity to the left; if the sensor is on the left, the fuzzy controller corrects the angular velocity to the right, as can be observed in Figure 19.

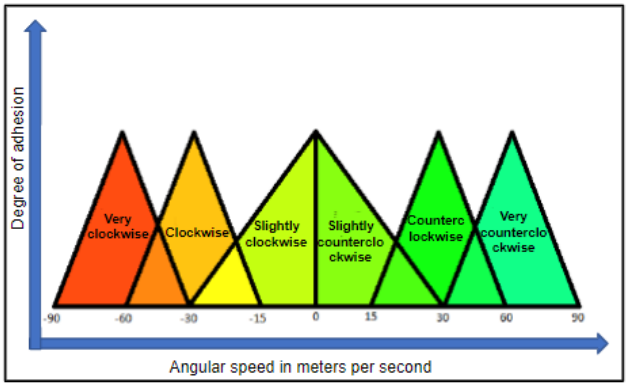


Fig. 19. Output Membership Functions: Angular Velocity

The rules for angular velocity control are intuitive; some examples include: if very far right, correct slightly counterclockwise; if far left, correct counterclockwise; if center right, correct significantly counterclockwise. The rules for angular velocity control are represented in Figure 20.

The obstacle avoidance control logic uses all the rules mentioned earlier in a fuzzy controller, which is activated only when an obstacle is encountered in the AGV's path. While the laser scanner does not detect an obstacle in the deviation region, the AGV is controlled by the PID controller. In this way, the control priority of the AGV is with the fuzzy controller, and after the obstacle avoidance is completed, the PID controller resumes operation.

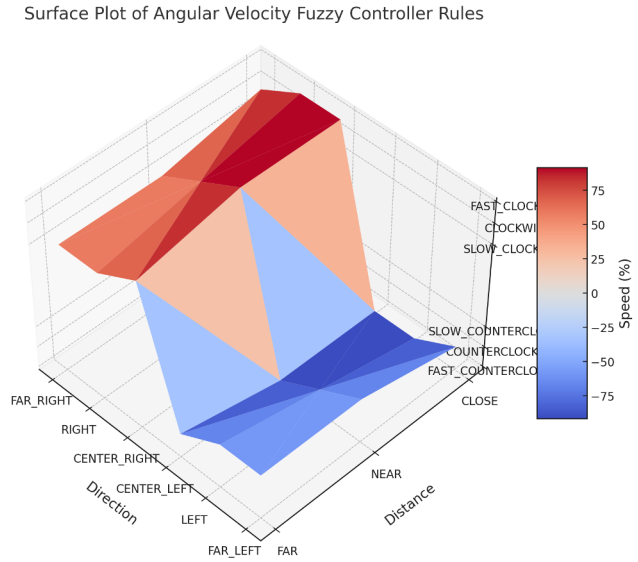


Fig. 20. Rules for Angular Velocity Control

III. Results and Discussion

Figures 21 and 22 , respectively, depict the final modeling of the production line used in the project and the G3R1000 AGV developed in Fusion 360 software.

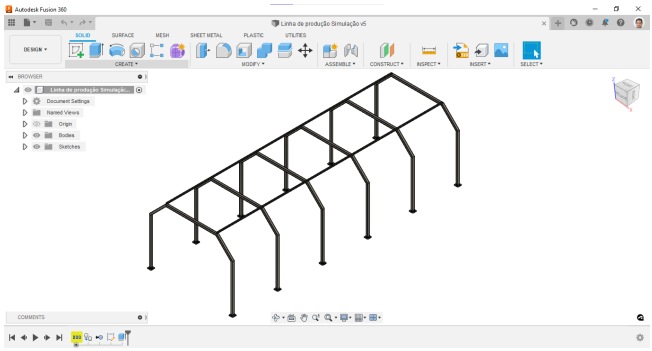


Fig. 21. Production Line Modeling

Regarding the scene projected in CoppeliaSim, the top view of the AGV is shown in Figure 23, the bottom view in Figure 24, and the complete scene in Figure 25.

After the correct parameterization of the control variables, both for the PID controller, including kp, ki, and kd, and for the membership functions of the fuzzy controller, the AGV was able to perform navigation and reach the planned goal, avoiding fixed obstacles as can be observed in Figure 26, as well as random obstacles shown in Figure 27.

The results demonstrate that using a Digital Twin for developing and validating AGV control algorithms in industrial environments is effective. Detailed modeling of the AGV G3R1000 and the production line in Fusion 360, followed by simulation in CoppeliaSim, allowed for a comprehensive

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.

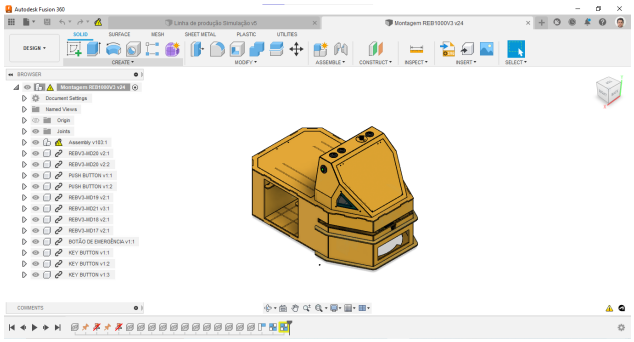


Fig. 22. AGV Modeling

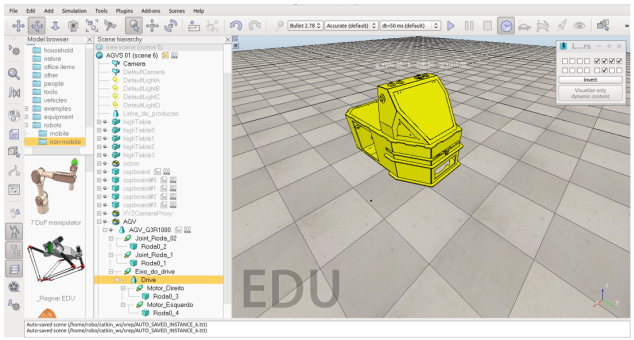


Fig. 23. Assembled AGV

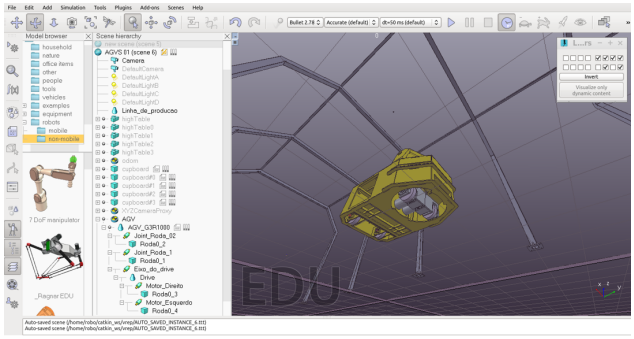


Fig. 24. Bottom view of the assembled AGV

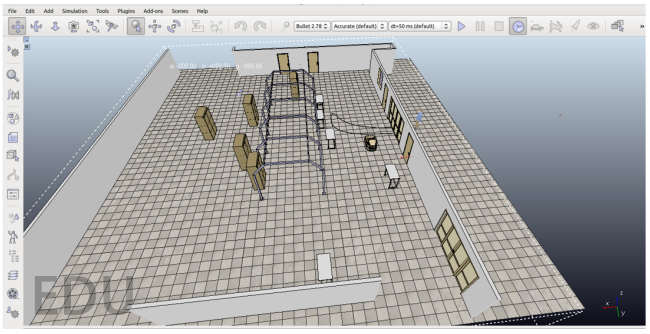


Fig. 25. Complete scene

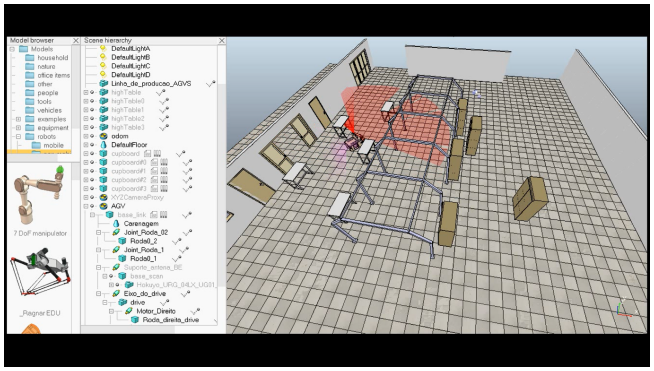


Fig. 26. Avoidance of fixed obstacles

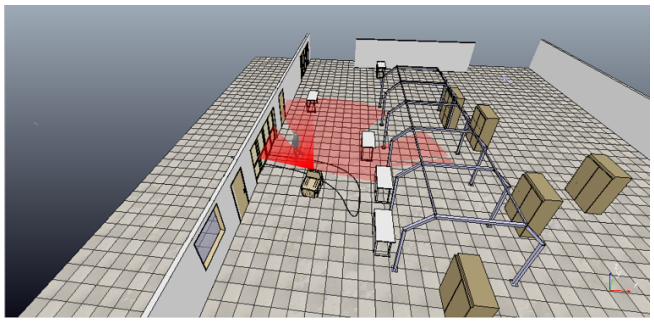


Fig. 27. Avoidance of moving obstacle

analysis of the robot's behavior. The key advantages include cost and time reduction, as virtual testing eliminates the need for physical resources until the algorithms are optimized. This leads to significant savings and enhances operational safety by identifying and mitigating risks before real-world implementation. The approach also offers flexibility and scalability, facilitating continuous adaptation and refinement of algorithms. Integration with the Robot Operating System (ROS) allows for a smooth transition from simulation to the physical robot.

The Fusion 360 software used for three-dimensional modeling of the equipment proved to be suitable and easy to use, as did the CoppeliaSim simulation software, which provided all the necessary tools to create a simulation environment similar to the real world. However, CoppeliaSim revealed significant inefficiency in terms of computational cost, requiring substantial processing power even when using native system components. On the other hand, ROS proved to be an effective tool, easy to use, and capable of seamless integration with real robots. Challenges include the complex parametrization of PID and Fuzzy controllers, which requires fine-tuning and specialized expertise, as well as the high computational demands of CoppeliaSim simulations. Despite these challenges, the successful application of control algorithms in simulation and their validation on the real robot show that mobile robotics concepts can be effectively applied in industrial settings.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.

Regarding the PID controller, its effectiveness in AGV navigation in a simulated environment was evident, as only a few adjustments to the kp, ki, and kd parameters were required. As for the fuzzy controller, besides being easy to implement, the language used to describe its rules makes it understandable and intuitive. However, the parametrization of the functions proved to be a challenge, and this aspect needs improvement for better utilization of the tool.

Furthermore, during the work, we identified a crucial point: the laser scanner was fixed to the AGV's structure, and the vehicle used a differential drive system, as shown in Figure 18. This resulted in the need to calculate the transformation of the laser scanner's angle concerning the differential drive system's angle of the AGV, as detailed in Figure 19. In summary, regarding the main objective of this work, we conclude that the AGV's behavior in the simulation scenario resembled real-world behavior, especially when programmed using ROS. Therefore, through the integration of ROS, the publication and reading of topics in CoppeliaSim, along with the use of the PID controller for navigation and the fuzzy controller for obstacle avoidance, it becomes evident that mobile robotics concepts can be applied as a strategy for autonomous movement in a simulated environment. Ultimately, the utilization of this tool in industrial environments, in a manner similar to the real world, is feasible and promising.

IV. Conclusion

In conclusion, Digital Twins offer an innovative and efficient strategy for developing automation technologies in Industry 4.0. By enabling detailed simulations of AGVs and their environments, they reduce costs and implementation time while enhancing operational safety. The integration of tools like Fusion 360, CoppeliaSim, and ROS facilitates a smooth transition from simulation to real-world application, proving beneficial for industrial use.

Future work can explore new simulation techniques, improve control algorithms, and expand the application of Digital Twins to various mobile robots and environments. Advances in computational power and simulation software, along with refined parametrization of PID and Fuzzy controllers, will further enhance the effectiveness and accessibility of this approach. Expanding Digital Twin applications to different robotic systems will allow for higher customization and optimization, improving performance across various industrial scenarios. As Industry 4.0 evolves, Digital Twins will continue to drive innovation and efficiency in automated systems.

References

[1] Schwab, K. (2017). The Fourth Industrial Revolution. Currency.

[2] VDI/VDE. (2018). VDI/VDE 2193 Blatt 1: Cyber-Physical Systems – Reference Architecture Model Industrie 4.0 (RAMI4.0).

[3] Tao, F., Qi, Q. (2019). "Make more digital twins." Nature, 573, 490–491.

[4] Qi, Q., Tao, F. (2018). "Digital Twins and Big Data Towards Smart Manufacturing and Industry 4.0: 360 Degree Comparison." IEEE Access, 6, 3585–3593.

[5] Rosen, R., Wichert, G.V., Lo, G., Bettenhausen, K.D. (2015). "About The Importance of Autonomy and Digital Twins for the Future of Manufacturing." IFAC-PapersOnLine, 48(3), 567–572.

[6] Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W. (2018). "Digital Twin in manufacturing: A categorical literature review and classification." IFAC-PapersOnLine, 51(11), 1016–1022.

[7] E. Rohmer, S. P. Singh, and M. Freese, "V-REP: A versatile and scalable robot simulation framework", in *Intelligent Robots and Systems (IROS), 2013 IEEE/RSJ International Conference on*. IEEE, 2013, pp. 1321–1326.

[8] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System", in *ICRA workshop on open source software*, vol. 3, no. 3.2. Kobe, 2009, p. 5.

[9] Ferreira, T., Garcia, O., Vaqueiro J., "Software Architecture for an autonomous car simulation using ROS, Morse and a QT based software for control and monitoring", *SBAI - Simpósio Brasileiro de Automação Inteligente*, 2015.

[10] Passino, K. Yurkovich, S. "Fuzzy Control", Addison-Wesley, California, USA, 1998.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:25:57 UTC from IEEE Xplore. Restrictions apply.